

Contents

SPECIFICATION — Minimal Coding Agent (closed-loop control, context engineering, tool use, permission gate, verification, trajectory instrumentation, + uv)	2
0. Intent and purpose	3
1. Actors and goals	5
2. Requirements (intent, high level)	7
3. Behavior and state model	11
3.1 Lifecycle scope	11
3.2 The control loop (one run)	11
3.3 The instrumented-artifact pipeline	12
4. Interfaces / contracts	12
C-01 Task specification	12
C-02 Policy interface	13
C-03 Tool schema (pinned; closed set)	13
C-04 Permission decision	13
C-05 Verifier specification + verdict	14
C-06 <code>trajectory.json</code> artifact (versioned, R-17)	14
C-07 <code>experiment.json</code> artifact (versioned, R-17)	16
C-08 Stopping conditions and <code>final_outcome</code> closure	16
C-09 §17 fixture (pinned, F-005)	17
C-08-bis Sandbox creation + lifecycle protocol (F-011/F-016) . .	18
5. Interface specification	18
5.1 CLI — primary surface (<code>agent</code> , R-16)	18
5.2 GUI	19
6. Invariants (must hold in every valid implementation)	19
7. Constraints (precise and measurable)	21
8. Edge cases and failure semantics	22
9. Acceptance criteria, tests, and evals	24
9.1 Closed-loop correctness	24
9.2 Failure injection (§17 / R-11)	25
9.3 Termination and instrumentation	25
9.4 Determinism and offline core	25
9.5 Model-availability taxonomy	25
9.6 Compaction and context engineering	26
10. Dependencies and environment	26
11. Traceability matrix (<code>id</code> → where realized)	26

SPECIFICATION — Minimal Coding Agent (closed-loop control, context engineering, tool use, permission gate, verification, trajectory instrumentation, + uv)

- **Status:** v0.2 — SPEC_REVIEW P0 (F-001...F-005) and P1 (F-006...F-011, F-013, out-of-scope) findings integrated inline (per SPEC_REVIEW_REPORT.md); targeting **Level 3**. P2 hygiene (F-012 ... F-016) deferred.
- **Language:** Python 3.12 | uv | Agent harness: deterministic loop + MockPolicy (offline double) / Ollama qwen3.8 over <http://localhost:11434> (opt-in real) | Environments: sandboxed working-tree + shell + test runner | Instrumentation: JSON trajectory log | GUI: none (MAY).
- **Curriculum source:** curriculum/week3/chapter15.md (§1 The Coding Agent as a Control System, §2 The Agent Harness, §3 Context Management, §4 Tool Use, §5 Planning, §6 Execution, §7 Verification, §8 Feedback and Iteration, §9 Compaction, §10 Subagents, §11 Permissions, §12 The Repository as the Agent's Environment, §13 Why Tests Become Part of the Agent's Reasoning System, §14 Coding Agents as Search Systems, §15 The Full Architecture, §16 What Actually Determines Coding-Agent Quality, §17 Build a Minimal Coding Agent, §18 Key Takeaways).
- **Scope of this document:** the *authoritative specification* of a **minimal coding agent** (ch15 §17/§15) — the *harness* that embeds an LLM-driven policy inside a closed-loop controller (observe → reason → act → verify → feedback → repeat, §1), with **context management**, a **tool controller**, a **permission layer** enforced *outside* the model (§11), a **verifier** that closes the loop (§7/§13), and a **trajectory instrumentation** layer that turns one run into a measurable record (§17). It is written to Level 2–3: behavior, interfaces, invariants, edge cases, and failure semantics are made explicit so an agent (or engineer) can derive implementation **and** verification.
- **Normative language:** MUST, MUST NOT, SHALL, and SHALL NOT are normative. SHOULD denotes a strong recommendation; MAY an optional behavior.
- **Principle:** ch15 §16 — **system architecture can amplify model capability**; two agents built on the *identical* model diverge in capability by their *engineering loop* (System A LLM → patch → return vs System B's full observe/inspect/search/verify/repair loop, §16). The central conceptual transition the exercise (ch15 §17/§18.7) demands:

move from “*The model generated code*” to “*The system successfully navigated a software environment to a verified state.*” The agent is a **closed-loop controller** ($A_t \sim \pi_\theta(A \mid C_t, O_t)$, $S_{t+1} = T(S_t, A_t)$, §1); the model supplies the policy, the harness supplies the control machinery, the repository is the environment, and verification supplies the feedback.

0. Intent and purpose

Chapter 15’s central lesson is the mechanism that makes an LLM *engineerable* as a coder:

A coding agent is not an LLM. It is an LLM embedded inside a runtime that provides context, tools, state, permissions, execution, and verification (ch15 §18.1, §18.3). Its fundamental cycle is **Observe → Reason → Act → Verify → Feedback → Repeat** (§18.2). The quality of an agent is not a function of its model alone but of a composition: **Coding Agent = Model + Harness + Tools + Context + Verification + Control** (§15), or more completely $Q_{\text{agent}} = f(Q_{\text{model}}, Q_{\text{context}}, Q_{\text{tools}}, Q_{\text{planning}}, Q_{\text{verification}}, Q_{\text{recovery}}, Q_{\text{permissions}})$ (§16).

This lab specifies the §17 exercise (*Build a Minimal Coding Agent*): build a small coding agent around a repository whose **objective is not** to produce a production autonomous programmer but to **understand the control loop**. The agent **MUST** support the ten-step §17 pipeline — receive a task, inspect files, search the codebase, read relevant files, propose an implementation, modify files, run tests, read failures, iterate, and **stop when verification succeeds** — arranged over the minimal §17 architecture **Task → Context builder → LLM → Tool selection → Permission check → Tool execution → Verifier → Feedback** (loop).

The lab instantiates ch15 §15’s full architecture as **one deterministic boundary and one probabilistic boundary**, the same reliability split used by week 1:

- **Deterministic boundary (pure, offline, reproducible, no LLM, no network):** the **control loop**, the **context manager** (what to include in C_t , §3), the **tool controller** (tool definitions + execution, §4/§6), the **permission layer** (§11), the **verifier runner** (tests/typecheck/lint, §7), the **stopping-condition logic** (§8), and the **trajectory logger** (§17). Offline, the *policy* itself is a deterministic **MockPolicy** (a scripted sequence of actions, or a rule-driven repair policy) so the whole loop can be re-run by CI without any model.
- **Probabilistic boundary (unreliable component):** the real **Ollama policy** (qwen3.8) that emits tool-call selections and edits. It is isolated

behind the `Policy` interface (§4) and replaced by the `MockPolicy` double for the automated suite. The model-availability taxonomy `DEGRADED MOCK / PULL_REQUIRED / RUN_REAL` (carried forward from week 1 eval/rag work) resolves a missing daemon so a *run degrading to the mock policy* is never ambiguous in the trajectory.

The evaluation discipline (§8/§13). The agent’s *own* acceptance signal is the **verifier**: tests, a type checker, a linter, a build, or a repo-specific check (§7) transform uncertain model output into measurable feedback. Per ch15 §13, tests are not merely a final gate — they are part of the agent’s *reasoning environment*: their output is fed back into the next **observe** step and guides subsequent actions. Per ch15 §14 the agent is a **search-and-control system**: it explores a space of possible repository states and uses verification to *eliminate* infeasible trajectories.

The instrumented experiment (§17/§8). The core acceptance activity is to **intentionally introduce a failure** — *Task: add a function that parses configuration; Experiment: inject an incorrect implementation* — and *observe* (a) can the agent **detect** the failure, (b) can it **diagnose the cause**, (c) can it **repair** the implementation, and (d) **how many iterations** it required. Each run is fully instrumented (§17) over the `trajectory.json` fields `iteration` / `tool_calls` / `tokens` / `files_read` / `files_modified` / `tests_executed` / `test_results` / `errors` / `time_per_iteration` / `final_outcome`. The experiment is reproducible *because* the `MockPolicy` (or a fixed seed) deterministically walks the detection→diagnosis→repair arc. The **fixture, canonical defect, and VerifySpec for this experiment are pinned in C-09 (F-005)** so that T-04/T-11 assert *specific, byte-reproducible* numbers rather than “whatever the model did today”.

Relationship to prior weeks. This lab is the *agent-week* payoff of the reliability split introduced in week 1: the deterministic-vs-probabilistic seam, the `MockXxx` doubles, the model-availability taxonomy, and “verification is the bridge between probabilistic behavior and engineering discipline” all carry forward. The verifier here subsumes week 1’s evaluator in *spirit* (a behavioral contract that the probabilistic system must satisfy, $f(x) \setminus \text{in } Y_{\text{acceptable}}$), but operates *inside the loop* as a runtime feedback signal rather than as an offline report. No upstream lab artifact is consumed by import: the agent is self-contained; it operates on an arbitrary target **repository sandbox** supplied per run.

Scope boundary / out of scope (F-p1 out-of-scope). This is a *minimal, single-agent* harness: one policy, one thread, one action batch per iteration. **Subagents / hierarchical deception (§10), multi-agent orchestration, persistent memory between runs, and remote/cloud executors are explicitly out of scope** — §10 is *referenced* for completeness but is not specified. No GUI is in scope by default (R-18’s `--baseline` and the MAY GUI are out, §5.2). The `OllamaPolicy` real path is specified only *by interface*; its behavior is best-effort and never asserted by the acceptance suite (K-06).

Primary product surface: a CLI (**agent**) with subcommands **run** (execute one task on a target repository and emit **trajectory.json** + a human summary), **experiment** (the §17 detection/ diagnosis/repair experiment over a target repo with an injected failure, emitting **experiment.json**), **inspect** (load a saved **trajectory.json** offline), and **compare** (regression Δ report over two artifacts, F-006). All durable artifacts (**trajectory.json**, **experiment.json**, **compare_report.json**) are written by one **report.py**; no subcommand prints its own ad-hoc serialization.

1. Actors and goals

Actor	Goals
User (human, single process)	Supply a coding task (task file or --task string) against a target repository (a local clone / sandbox, never the bootcamp repo itself); run the agent offline (mock policy) or against Ollama (--real); launch the §17 failure-injection experiment; read the instrumented trajectory and experiment report; optionally inspect a saved trajectory. (single-principal — no inter-principal authorization; permissions are <i>action-scoping</i> , not <i>user-scoping</i> , ch15 §11.)
Policy (policy.py)	The LLM-driven policy $\pi_{\theta}(A \mid C_t, O_t)$ (§1): given the current context C_t and observation O_t , select the next action A_t (a tool call / an edit / “stop”). Two implementations: MockPolicy (deterministic, offline — a scripted or rule-driven action sequence, the CI double) and OllamaPolicy (opt-in real, qwen3.8). Only policy.py calls the model (§11 analog of the core-import discipline, R-15).

Actor	Goals
ContextManager (<code>context.py</code>)	Compose C_t from the task, the trajectory-so-far, and the repository state (§3) <i>without</i> blindly passing the entire codebase (SHOULD -level context engineering); implement compaction (§9) — preserve salient state, discard irrelevant trajectory history — behind the token budget. Pure, deterministic.
ToolController (<code>tools.py</code>)	Define and dispatch the tool set (§4/§6): list_files , read_file , search , edit_file , run_shell (tests / typecheck / lint / build). Each tool has a fixed schema and executes inside the target repo sandbox only ; no tool may escape the sandbox root.
PermissionLayer (<code>permissions.py</code>)	Gate <i>every</i> action before execution (§11): map each tool call to an authorization decision (ALLOW / DENY) from an explicitly declared policy — a static allow-list of tools/operations, a dynamic check (path in-sandbox, command prefix), and MAY a rule-based policy. Enforced outside the model — the policy may <i>request</i> an action; only the permission layer <i>permits</i> it (ch15 §18.10).
Verifier (<code>verifier.py</code>)	After each modification, run the repository’s verification (§7): tests, type checker, linter, build, and MAY repo-specific / human checks. Close the loop by returning a structured verdict VERIFIED / FAILED + captured output that feeds the next observe (§8). The verifier is <i>part of the reasoning environment</i> (§13), not merely a final gate.

Actor	Goals
ControlLoop (<code>control_loop.py</code> , F-011)	The runtime harness (§2): wire policy → permission → tool → verifier → feedback , drive the observe/reason/act/verify/feedback state machine (§3.1/§3.2), enforce stopping conditions — verification succeeds, or a bounded --max-iterations cap is hit (§8) — and never loop forever (I-001).
TrajectoryLogger (<code>instrument.py</code>)	Record every iteration’s §17 fields (iteration , tool_calls , tokens , files_read , files_modified , tests_executed , test_results , errors , time_per_iteration) and the final_outcome (§17/§14). Byte-deterministic on the mock path (I-002).
Target repository / sandbox (<i>per-run input</i>)	The environment S_t (§1): source files, config, dependencies, build artifacts, test results, git state, environment. It is <i>copied into</i> an ephemeral sandbox so the agent’s edits never mutate the user’s working tree or the bootcamp repo; the verifier runs <i>inside</i> the sandbox.
Ollama daemon (<i>external, opt-in</i>)	Local runtime at http://localhost:11434 : real generation/tool-call selection. Not part of this project; the model-availability taxonomy (R-15) resolves its absence.

2. Requirements (intent, high level)

ID	Statement
R-01	The system executes the ch15 §17 minimal coding-agent pipeline in a closed loop — <i>receive task</i> → <i>inspect repository files</i> → <i>search the codebase</i> → <i>read relevant files</i> → <i>propose implementation</i> → <i>modify files</i> → <i>run tests</i> → <i>read failures</i> → <i>iterate</i> → stop when verification succeeds — arranged over the §17 architecture Task → Context builder → LLM → Tool selection → Permission check → Tool execution → Verifier → Feedback (loop). The loop is the §1/§18.2 controller observe → reason → act → verify → feedback → repeat ($A_t \sim \pi_\theta(A \mid C_t, O_t)$, $S_{t+1} = T(S_t, A_t)$); it terminates only on a defined stopping condition (R-08), never by accident.
R-02	The system is an agent harness (§2), not an LLM: it wires policy → permission → tool → verifier → feedback and owns tool definitions, tool execution, and state management across iterations (§2). The LLM is reached <i>only</i> through the Policy interface (R-15); the harness itself contains no model call.
R-03	Context engineering (§3/§18.4): the ContextManager composes C_t dynamically from the task, the trajectory-so-far, and observed repository state O_t, selecting the <i>relevant</i> slice of the repository rather than passing the entire codebase (C_t MUST NOT be the whole repo by default; I-005). Context composition MUST be deterministic for identical inputs.
R-04	Tool use (§4/§6): the ToolController dispatches a fixed, schema-pinned tool set — list_files, read_file, search, edit_file, run_shell (running tests / typecheck / lint / build) — each with a declared input/output contract (C-03). Tool execution extends the model into the filesystem/shell/verification <i>environment</i> (§18.5).
R-05	Permissions enforced outside the model (§11/§18.10): <i>every</i> tool call is gated by the PermissionLayer before execution (§3.2 act-stage) via a static allow-list (tools/operations), a dynamic check (path-in-sandbox, command-prefix), and MAY a rule-based policy; the decision is ALLOW/DENY (C-04). The policy may only <i>request</i> an action; only the permission layer <i>permits</i> it.
R-06	Verification closes the loop (§7/§13/§18.11): after each modification the Verifier runs the repository’s verification — tests, type-checker, linter, build, and MAY repo-specific / human checks — and returns a structured verdict VERIFIED/FAILED plus captured output (C-05). The verifier output is a reasoning signal fed into the next observe step, not merely a final gate (§13).

ID	Statement
R-07	Feedback and iteration (§8): a FAILED verdict is fed back as the next <code>0_t</code> and the loop continues (detection → diagnosis → repair). Per ch15 §14 the agent is a search-and-control system : each iteration eliminates infeasible trajectories by discarding states that fail verification.
R-08	Stopping conditions (§8): the loop stops when (a) the verifier returns VERIFIED (<code>final_outcome = VERIFIED</code>), or (b) a bounded <code>--max-iterations N</code> cap is reached (<code>final_outcome = BUDGET_EXHAUSTED</code> , a <i>non-zero</i> exit), and MAY stop on repeated identical consecutive trajectories / no-op actions (STALLED). The default <code>--max-iterations</code> is finite (I-001 : no unbounded loop).
R-09	Trajectory instrumentation (§17/§14): every run emits <code>trajectory.json</code> recording, <i>per iteration</i> , the §17 fields <code>iteration</code> , <code>tool_calls</code> , <code>tokens</code> (consumed), <code>files_read</code> , <code>files_modified</code> , <code>tests_executed</code> , <code>test_results</code> , <code>errors</code> , <code>time_per_iteration</code> , and a top-level <code>final_outcome</code> . Emitted by <code>report.py</code> (R-16); byte-deterministic on the mock path (I-002).
R-10	Compaction (§9, MAY/default-on): for trajectories exceeding a token budget the <code>ContextManager</code> compacts — preserving salient state (task, open edits, last VERIFIED/FAILED verdict, file manifest) and discarding redundant history — so a long agent run does not blow the context window. Compaction is state management , not merely token trimming (§18.8).
R-11	The ch15 §17 failure-injection experiment is a fixed acceptance activity (T-04/T-05): <i>Task</i> — <i>add a function that parses configuration</i> ; <i>Experiment</i> — <i>inject an incorrect implementation</i> . The run MUST demonstrate the detection → diagnosis → repair arc and report the number of iterations required; it MUST be reproducible on the <code>MockPolicy</code> path.
R-12	Sandbox isolation (§12/§16): each run operates on a <i>copy</i> of the target repository in an ephemeral sandbox root; no tool or the verifier ever mutate the user’s working tree or the bootcamp repo, and no <code>run_shell/search/read_file/edit_file</code> may resolve a path <i>outside</i> the sandbox root (R-05, I-003). The sandbox is the agent’s environment <code>S_t</code> (§12/§18.12).

ID	Statement
R-13	Offline determinism (carried week-1 discipline): the <i>entire automated suite</i> runs offline via <code>MockPolicy</code> — a scripted or rule-driven policy — with no network and no model ; mock-path artifacts are byte-identical for identical inputs (I-002). <code>tokens/time_per_iteration</code> on the mock path are deterministic surrogates, explicitly labeled synthetic in the trajectory (E-04). The real Ollama path is opt-in/manual and best-effort.
R-14	Model-availability taxonomy (carried): on <code>--real</code> start the agent resolves <code>DEGRADED MOCK</code> (daemon unreachable → <code>MockPolicy</code> + banner, exit 0), <code>PULL_REQUIRED</code> (<code>qwen3.8</code> not pulled → remediation string + exit 4), or <code>RUN_REAL</code> (pulled → real policy, no banner, exit 0). Each outcome carries a distinct banner so a <i>run that degraded to the mock policy</i> is never ambiguous in the trajectory.
R-15	Policy isolation / determinism boundary (§2/§11 analog of core-import discipline): the deterministic harness core (<code>control_loop.py</code> , <code>context.py</code> , <code>tools.py</code> , <code>permissions.py</code> , <code>verifier.py</code> , <code>instrument.py</code> , <code>report.py</code>) MUST be LLM- and network-free , asserted by an import/graph source scan (T-02). Only <code>policy.py</code> (the <code>OllamaPolicy</code> branch) MAY import the network/model client; <code>MockPolicy</code> does not.
R-16	The primary product surface is the CLI agent with subcommands <code>run</code> , <code>experiment</code> , <code>inspect</code> (§5.1). All durable artifacts (<code>trajectory.json</code> , <code>experiment.json</code>) are written by one <code>report.py</code> ; no subcommand prints its own ad-hoc serialization.
R-17	Schema gate (carried): <code>trajectory.json</code> and <code>experiment.json</code> carry a literal <code>trajectory_version/experiment_version == "0.1"</code> field; <code>inspect/experiment</code> refuse a mismatched version unless <code>--force</code> (E-06). A malformed artifact failing validation is a deterministic load error (E-05).
R-18	System-A vs System-B demonstration (ch15 §16, SHOULD): the CLI MAY run a <code>--baseline</code> “System A” (patch-only: <code>read task</code> → <code>propose</code> → <code>write</code> → <code>stop</code> , no <code>verify/loop</code>) alongside the full “System B” to make §16’s thesis observable — same task, same (mock) model, different loop, divergent capability — and record both trajectories. Not required for acceptance.

3. Behavior and state model

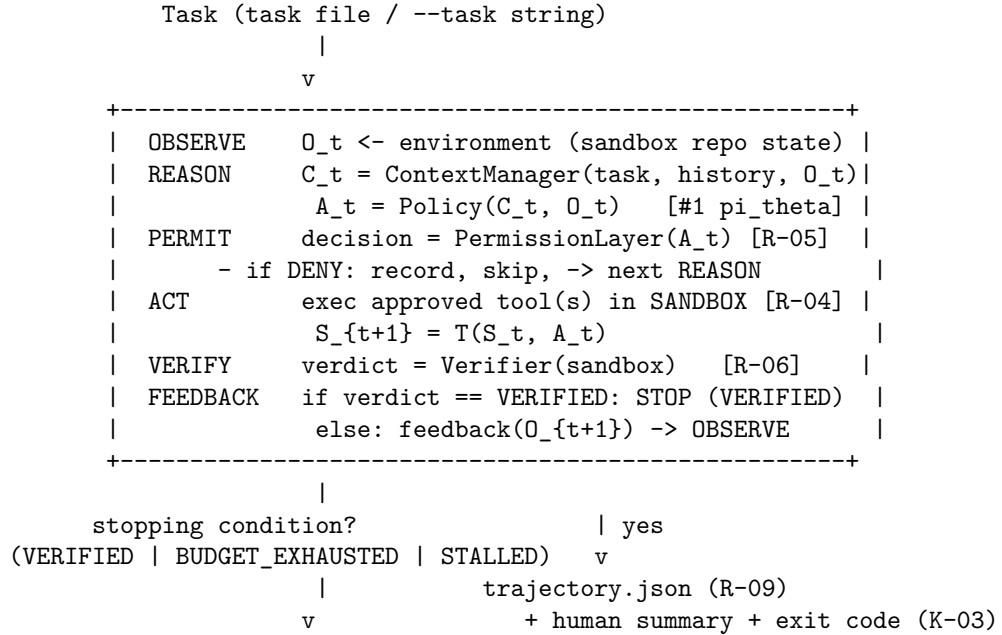
3.1 Lifecycle scope

One **run** is one closed-loop task execution. Each run moves through a finite state machine whose drive is the §1 controller; the *durable artifact* is the trajectory over that run, not the loop’s transient memory. Within one run, three nested scopes operate:

- **Per-iteration** (OBSERVE → REASON → ACT → VERIFY → FEEDBACK): compose **C_t**, let the Policy pick **A_t**, gate **A_t** through the permission layer, execute the approved tool(s) in the sandbox, run the verifier, capture the verdict + feedback, advance **t**. Each iteration is one row of the §17 instrumentation.
- **Per-run** (whole loop): accumulate iterations until a **stopping condition** (R-08) fires; emit **trajectory.json** + a human summary.
- **Per-experiment** (experiment subcommand, §17): wrap one or more runs over the injection→repair scenario; emit **experiment.json** summarizing the detect/diagnose/repair arc and the iteration count.

The **deterministic/probabilistic seam**: context, permissions, tools, verifier, and loop are deterministic; only policy (its OllamaPolicy branch) is probabilistic. On the mock path no probabilistic component participates (R-13).

3.2 The control loop (one run)



The agent moves through states `RECEIVED → OBSERVING → REASONING → PERMITTING → ACTING → VERIFYING → FEEDBACK`, cycling back to `OBSERVING` on any `FAILED` verdict, and settling in a terminal state `{VERIFIED, BUDGET_EXHAUSTED, STALLED:NOOP, STALLED:BUDGET, DENIED_LOOP, ERROR}` (F-004 split). A `DENY` at the `PERMIT` stage does **not** end the run — it routes to the next `REASON`; a *repeated* K-08-denied cycle reaches `DENIED_LOOP`, and a *repeated* `NOOP` action (the `NOOP` transition of F-015, count via `--max-consecutive-noops`, default = `--max-iterations / 2`) reaches `STALLED:NOOP`. A `--no-compact` context overflow reaches `STALLED:BUDGET` (E-13). The loop is always **bounded**: a run terminates within `--max-iterations+1` iterations (I-001) and by at most one of the non-`VERIFIED` terminals above.

3.3 The instrumented-artifact pipeline

The agent’s *only* durable artifacts are files written by `report.py`:

- `trajectory.json` (from `run`) — the versioned, schema-gated record (C-06): per-iteration rows + top-level `final_outcome`, `policy` (mock/ollama), model-availability banner (R-14), and the §17 field set.
- `experiment.json` (from `experiment`) — the versioned record (C-07): the injected-failure scenario, per-run `detect/diagnose/repair` phase markers, the iteration-to-`VERIFIED` count, plus the embedded `trajectory.json` ref.

All downstream readers (`inspect`, the optional `--baseline` System-A/B compare, R-18) read **only** these artifacts — they never re-run the loop or re-touch the sandbox (I-006).

4. Interfaces / contracts

C-01 Task specification

```
# task.py - the input to one run
@dataclass
class Task:
    task_id: str          # stable id, e.g. "parse-config"
    prompt: str           # natural-language task (the --task string or task file b
    target_repo: str      # path to the repo to be copied into the sandbox
    verifier: "VerifySpec" # C-05: which verification closes the loop
    success_token: str | None = None # MAY: a substring/regex the repaired artifact must
    acceptance_test: str | None = None # MAY: a specific test id that must pass to count as
```

C-02 Policy interface

```
# policy.py - the ONLY module that MAY call the model/network (R-15)
class Policy(Protocol):
    def select(self, context: Context, observation: Observation) -> Action: ...

# Action is a closed tag-union (I-004):
#   ToolCall(name: str, args: dict)      # -> routed to PermissionLayer then ToolController
#   STOP(final_outcome: "VERIFIED")      # policy may declare completion, but only a
#                                       # VERIFIED verifier may settle the run (see loop)
#   NOOP(note: str)                     # -> feeds STALLED detection (R-08)
```

MockPolicy is deterministic: it is a **scripted** list of Actions (for fixed acceptance experiments) or a **rule-driven** policy (read→inspect→search→edit→verify→repair rules, for the injection experiment). OllamaPolicy wraps the model with a fixed **seed** where supported; otherwise it is labeled **seed=None** and excluded from byte-identity (I-002 applies to the mock path only).

C-03 Tool schema (pinned; closed set)

```
# tools.py - each tool is a (name, input schema, output schema, sandbox-bound) callable
TOOL_SET = {
    "list_files": {"in": {"path": str, "glob": str?}, "out": "list[str]"},
    "read_file": {"in": {"path": str}, "out": "str"},
    "search": {"in": {"query": str, "path_glob": str?}, "out": "list[Hit]"},
    "edit_file": {"in": {"path": str, "op": enum[replace|append|prepend], "old"?: str, "new": str},
                  "out": "EditResult{applied: bool, diff: str}"},
    "run_shell": {"in": {"command": str, "cwd": str?}, "out": "ProcResult{exit_code: int, output: str}"},
}
```

Every tool resolves its **path/cwd** strictly *inside* the sandbox root (R-12, I-003). **edit_file** is the only mutation tool; **run_shell** is used for *verification* (tests/typecheck/lint/build) and is itself permission-gated.

edit_file failure semantics (F-010, E-14). **edit_file** with **op="replace"** returns **EditResult{applied: false, diff: ""}** when its old substring is not found in the target file; a failed edit is **not** silent: the **EditResult** (including a non-applied diff and the reason) is surfaced into the next iteration's **observation** so the repair policy can correct its **old/new**, and it does **not** advance **files_modified** (only **applied=true** counts, C-06 counting rules). A run in which **edit_file** fails on *every* attempt feeds the K-08 consecutive-failure budget and terminates **STALLED** (or the cap), never a false **VERIFIED** (§13: the verifier, not a failed edit, closes the loop).

C-04 Permission decision

```
# permissions.py - evaluated BEFORE any tool executes (R-05)
ALLOW = {"tool": str, "args": dict, "reason": "ALLOWED"}
```

```
DENY = {"tool": str, "args": dict, "reason": "NOT_IN_ALLOWLIST" | "PATH_OUTSIDE_SANDBOX"
        | "COMMAND_FORBIDDEN" | "RULE_DENIED", "detail": str}
```

```
def authorize(tool_call: Action, sandbox_root: str, pconfig: PermsConfig) -> "ALLOW | DENY"
```

PermsConfig declares: allow_list (tools/operations, static §11.1), sandbox_root + path check (dynamic §11.2), and MAY a rule function (policy-based §11.3). Evaluation precedence: (1) path-in-sandbox, (2) command-prefix allow for run_shell, (3) tool in allow_list, (4) MAY rule override. First DENY wins.

C-05 Verifier specification + verdict

```
@dataclass
class VerifySpec:
    kind: enum["tests", "typecheck", "lint", "build", "repo_specific"] # §7.1-5
    command: str # e.g. "pytest -q" / "python -m mypy" / "python -m ruff"
    success_exit: int = 0

@dataclass
class Verdict:
    status: "VERIFIED" | "FAILED" | "ERROR" # §7
    checks: list[dict] # per-check {kind, command, exit, output_tail}
    output: str # captured stdout/stderr tail; fed into the next OBSERVE (§13)
```

A run *settles* VERIFIED only when the verdict is VERIFIED for the run's VerifySpec (R-06). FAILED (non-zero exit, captured) and ERROR (verifier could not run, e.g. import failure / runner missing) are distinct: ERROR is reported but does not by itself terminate (it feeds the next iteration) unless it recurs (F-002: **K-08** consecutive-ERROR/DENY budget, not K-04). The §17 experiment's VerifySpec is pinned (F-005): VerifySpec(kind="tests", command="pytest -q", success_exit=0) over the C-09 fixtures/parse-config/ fixture, so T-04/T-11 assert concrete, reproducible verdicts.

C-06 trajectory.json artifact (versioned, R-17)

```
{
  "trajectory_version": "0.1",
  "task_id": "parse-config",
  "policy": "mock",
  "availability_banner": null, # set only on DEGRADED MOCK, else null
  "sandbox_root": "/tmp/agent-sbx/...",
  "iterations": [
    {
      "iteration": 1,
      "tool_calls": [{"name": "list_files", "args": {"path": "."}}, {"name": "read_file",
```

```

    "tokens": {"estimated": 1820, "mode": "synthetic"},
    "files_read": ["src/config.py"],
    "files_modified": [],
    "tests_executed": 0,
    "test_results": null,
    "errors": [],
    "time_ms": 42,
    "verdict": "PENDING",
    "phase": "observe"
  }
],
"final_outcome": "VERIFIED",
"iterations_used": 4,
"total_tokens": {"estimated": 9140, "mode": "synthetic"}
}

```

phase vocabulary (F-008, pinned). `phase` is the C-06 *instrumentation* label for the iteration's §3.2 FSM state, mapped by a fixed table so the field is schema-validatable (`schemas/trajectory.json`): `observe`↔`OBSERVING`, `inspect/search`↔`REASONING` (read/search sub-phases within reason), `propose`↔`REASONING`, `modify`↔`ACTING`, `verify`↔`VERIFYING`, `repair`↔`VERIFYING`→`REASONING` (a FAILED verdict routed back to reason), `stop`↔<terminal>. Each row's phase MUST be exactly one enum value from {`observe`, `inspect`, `search`, `propose`, `modify`, `verify`, `repair`, `stop`}.

The **win-rule for final_outcome (F-014)**: `final_outcome` is always the *terminal-stopping* outcome (C-08), **not** the last per-iteration verdict; on `BUDGET_EXHAUSTED`/`STALLED`/`DENIED_LOOP` it is that label regardless of the last verdict; a `final_outcome` of `ERROR` is reached *only* by the terminal-error path (E-02/E-03/K-08).

The §17 field set (`iteration`, `tool_calls`, `tokens`, `files_read`, `files_modified`, `tests_executed`, `test_results`, `errors`, `time_per_iteration` → `time_ms`, `final_outcome`) MUST be present on every iteration row (R-09). `tokens.mode` == "synthetic" when `policy` == "mock" (R-13/E-04).

Iteration index base (F-001). Iterations are **1-based**: "iteration" is the 1-based sequence position (`iteration` 1 is the first), and `iterations_used` is the count of iterations actually run (the position of the terminal iteration). C-07's `phases[].iteration` and all §9 prose counts (T-01/T-04/T-06) use the same 1-based convention. **This is the byte-identity-affected convention I-002 depends on.**

Field counting rules (F-013, pinned for byte-identity).

- `tool_calls` — *every* action requested, including DENYed calls and the terminal STOP/NOOP.
- `files_read` — **distinct paths opened by read_file**; `search/list_files` *discover* files but do **not** increment `files_read`.

- `files_modified` — **distinct paths with a successful `edit_file`** (`applied=true`); a failed edit (`applied=false`, C-03/E-14) does not increment it.
- `tests_executed` — the count of verification *checks* the **verifier** ran this iteration (0 when the `run_shell` runner itself failed, E-03); `test_results` is the captured pass/fail tallies, `null` when no check ran.
- `time_per_iteration` (`time_ms`) and `tokens` — on the mock path these are the **deterministic surrogate** (K-07), labeled `synthetic`; on the real path they are measured, labeled `measured`.

C-07 `experiment.json` artifact (versioned, R-17)

```
{
  "experiment_version": "0.1",
  "task_id": "parse-config",
  "injection": {"file": "src/config.py", "symbol": "parse_config",
               "injected_defect": "wrong split delimiter",
               "pre_injection_verdict": "FAILED"},
  "phases": [
    {"phase": "detect",   "iteration": 1, "evidence": "test_parse_basic FAILS"},
    {"phase": "diagnose", "iteration": 2, "evidence": "delimiter == '=' vs expected '=='"},
    {"phase": "repair",   "iteration": 3, "evidence": "edit_file applied to src/config.py"}
  ],
  "final_outcome": "VERIFIED",
  "iterations_to_verified": 3,
  "trajectory_ref": "trajectory.json"
}
```

The experiment **MUST** record detect/diagnose/repair (R-11) and the iteration-to-VERIFIED count. Its fixtures come from **C-09** (F-005): the canonical `parse-config` fixture, defect, and `VerifySpec` are pinned there so `iterations_to_verified` is a reproducible number, not a model-dependent one, and `phases[].iteration` are 1-based (F-001).

C-08 Stopping conditions and `final_outcome` closure

STOPPING (R-08):

VERIFIED	-> final_outcome VERIFIED	exit 0	
BUDGET_EXHAUSTED	-> final_outcome BUDGET_EXHAUSTED	exit 1	
STALLED:NOOP	-> final_outcome STALLED:NOOP	exit 1	(F-004, K-08)
STALLED:BUDGET	-> final_outcome STALLED:BUDGET	exit 1	(F-004, E-13; --no-compa
DENIED_LOOP	-> final_outcome DENIED_LOOP	exit 1	(F-004, K-08)
ERROR	-> final_outcome ERROR	exit 5	(F-003: partition; core-

Exit-code partition (F-003). The exit space is a **partition**: 0=VERIFIED · 1=BUDGET_EXHAUSTED/STALLED:*/DENIED_LOOP · 2=usage · 3=malformed-artifact load (inspect/compare) · 4=PULL_REQUIRED · 5=terminal core-

ERROR. A DENY does **not** terminate by itself (it routes to the next REASON); only a **K-08** consecutive-DENY cycle reaches DENIED_LOOP. E-02/E-03/E-10 are reconciled to this partition (core-ERROR/E-10 unwritable -> 5; PULL_REQUIRED stays exclusively 4).

C-09 §17 fixture (pinned, F-005)

Location: labs/week3/chapter15/fixtures/parse-config/ — a **versioned, pinned artifact** so T-04/T-11 assert *specific, reproducible* numbers (byte-identical under MockPolicy, I-002/I-013).

repo/config.py (the target source; parse_config is the unit under test):

```
def parse_config(raw: str, delimiter: str = "=") -> dict:
    """Parse `raw` key=value lines into a dict."""
    out = {}
    for line in raw.splitlines():
        if line == "" or delimiter not in line:
            continue
        k, v = line.split(delimiter, 1)
        out[k.strip()] = v.strip()
    return out
```

test/test_config.py (the verifier's pytest -q target):

```
def test_parse_basic():
    assert parse_config("a=1\nb=2") == {"a": "1", "b": "2"}

def test_ignores_blank_lines():
    assert parse_config("a=1\n\nb=2") == {"a": "1", "b": "2"}
```

The **canonical defect** (what MockPolicy injects and repairs): the blank-line filter hard-codes `line == ""` while the *split* uses `delimiter`, but the buggy variant *also* filters with a mismatched test so a non-key line is mis-parsed and `test_parse_basic` FAILS. The **repair** is one `edit_file`: fix the filter to `delimiter not in line` (a single-token change).

Pinned contract for the experiment:

Field	Value
task_id	parse-config
VerifySpec	kind="tests", command="pytest -q", success_exit=0
injected defect	parse_config mis-filters non-key lines
pre_injection_verdict	FAILED (T-04 iter 1, detect)
diagnosis	delimiter/filter mismatch (diagnose, T-04 iter 2, read_file/search)

Field	Value
repair	edit_file to repo/config.py (repair, iter 3, VERIFIED)
iterations_to_verified	3 (1-based, F-001)

This is the *only* fixture asserted numerically by T-04/T-11; other `--task` runs are free-form and not asserted on exact counts.

C-08-bis Sandbox creation + lifecycle protocol (F-011/F-016)

sandbox.py (owner added to the I-009 core list, F-011) creates the sandbox via `shutil.copytree(source, root, symlinks=False, dirs_exist_ok=False)` — **symlinks are not copied** (no escape vector, I-003) — and removes it in a `finally` block regardless of `final_outcome`. On allocation failure (unwritable/non-existent root) the run exits 5 (E-10, per the F-003 partition). `sandbox.py`, `control_loop.py`, `context.py`, `tools.py`, `permissions.py`, `verifier.py`, `instrument.py`, and `report.py` are the full deterministic-core module list (I-009/R-15).

5. Interface specification

5.1 CLI — primary surface (agent, R-16)

Subcommand	Behavior	Exit
agent run --task <file\ string> --repo <path> [--mock\ --real] [--max-iterations N] [--max-consecutive-errors M] [--no-compact] --out trajectory.json	One closed-loop run over a sandbox copy of --repo; emits trajectory.json (C-06) + a human summary. --max-iterations defaults to a finite cap (R-08); --max-consecutive-errors defaults via K-08.	0 VERIFIED / 1 BUD- GET_EXHAUSTED · STALLED · DENIED / 4 PULL_REQUIRED / 5 core-ERROR / 2 usage
agent experiment --task parse-config --repo <path> [--inject-defect <spec>] [--mock] --out experiment.json	The §17 failure-injection experiment (R-11/F-005): inject the C-09 canonical defect, run loop, record detect/diagnose/repair + iteration-to-VERIFIED count; emits experiment.json (C-07).	0 / 1 did-not- reach- VERIFIED / 4 PULL_REQUIRED / 5 core-ERROR / 2 usage

Subcommand	Behavior	Exit
<code>agent compare --baseline a.json --current b.json [--force]</code>	Regression Δ report over two versioned artifacts (F-006; <code>report.py</code> , I-006).	0 / 3 version-mismatch (E-06) / 2 usage
<code>agent inspect --in trajectory.json [--force]</code>	Load + render a saved <code>trajectory.json</code> offline (§17); refuse mismatched version unless <code>--force</code> (R-17/E-06).	0 / 3 malformed artifact / 2 usage

Usage errors (missing flags, bad paths, unknown subcommand) exit 2 consistently (K-01). The `--force` flag on `inspect/compare/experiment` bypasses the `*_version` mismatch gate (R-17/E-06). Global flags: `--verbose/--quiet` (log level), `--seed N` (Ollama reproducibility where supported; no-op on mock, recorded in the top-level `seed` field of C-06 when implemented, F-012 P2), `--sandbox <root>` (ephemeral sandbox root; default `$TMPDIR/agent-sbx/...`). The `--baseline` flag (R-18) MAY run a System-A patch-only trace alongside a System-B full-loop trace and diff their iteration/cost/tokens via the `compare` command (F-006). Exit codes follow the F-003 partition (0/1/2/3/4/5).

5.2 GUI

None by default (MAY). No GUI is specified at Level 1; a read-only trajectory browser MAY be added that **never runs inference or re-touches the sandbox** — it is read-only over the artifacts (the I-006 “downstream readers read *only* artifacts” property carries to the GUI). (F-007: the §5.1 version of this paragraph wrongly attributed the GUI to I-007, the verifier-signal invariant; the correct invariant is the I-006 artifact-only-read property.)

6. Invariants (must hold in every valid implementation)

ID	Invariant
I-001	Bounded loop / termination: the control loop runs strictly fewer than <code>--max-iterations+1</code> iterations; a run MUST terminate in { <code>VERIFIED</code> , <code>BUDGET_EXHAUSTED</code> , <code>STALLED:NOOP</code> , <code>STALLED:BUDGET</code> , <code>DENIED_LOOP</code> , <code>ERROR</code> } (F-004 split). No unbounded loop.

ID	Invariant
I-002	Byte-deterministic mock path: on MockPolicy over identical inputs, <code>trajectory.json/experiment.json</code> are byte-identical. Iteration order is fixed 1-based (F-001); per-iteration <code>tokens/time_ms</code> are the deterministic surrogates (K-07), labeled synthetic ; per-iteration arrays are emitted in iteration order. The Ollama path is excluded (opt-in, best-effort).
I-003	Sandbox isolation: every <code>list_files/read_file/search/edit_file/run_shell</code> resolves strictly inside the sandbox root; no tool may escape (<code>../</code> , symlinks, absolute, or <code>\$VAR</code>). Enforcement is in the tool layer <i>and</i> the permission layer (defense in depth).
I-004	Closed action/tool spaces: Action is the ToolCall STOP NOOP tag-union and TOOL_SET is the pinned C-03 set; an unrecognizable policy output is a deterministic ERROR action (E-02), never a silent no-op.
I-005	Context is selected, not bulk: C_t MUST NOT be the entire repository by default; only the task, recent feedback, and a dynamically-selected working set reach the policy. A context exceeding the token budget triggers compaction (R-10), not a pass-through.
I-006	Verdict-driven stop: the <i>only</i> path to a VERIFIED final_outcome is a VERIFIED Verdict for the run's VerifySpec. A policy-declared STOP is <i>accepted</i> as VERIFIED only when the verifier also returned VERIFIED; otherwise it is a FAILED-with-promise and the loop continues (or hits budget).
I-007	Verifier as a runtime signal, not just a final gate (§13): the verifier is invoked <i>after every modification</i> , not only at end; its captured output is written into the next iteration's C_t so subsequent actions read it.
I-008	Permission precedes execution: PermissionLayer.authorize is called <i>before</i> any tool side-effect; a DENY produces no side-effect and is recorded in that iteration's errors/tool_calls.
I-009	Deterministic core is LLM/network-free (R-15, F-011): the module list is fixed — <code>sandbox.py</code> , <code>control_loop.py</code> , <code>context.py</code> , <code>tools.py</code> , <code>permissions.py</code> , <code>verifier.py</code> , <code>instrument.py</code> , <code>report.py</code> — each of which MUST NOT import any model/network client. Only <code>policy.py:OllamaPolicy</code> MAY. Asserted by a source/graph scan (T-02) over exactly this list.

ID	Invariant
I-010	Trajectory field totality (§17): every <code>trajectory.json</code> iteration row carries exactly the C-06 field set; no row is dropped, none omits <code>final_outcome</code> .
I-011	Sandbox is ephemeral + non-recursive: edits land on the sandbox copy only; the agent’s loop cannot operate on its own source tree (a recursive self-edit would deadlock the test). The bootcamp repo is never a valid <code>--repo</code> .
I-012	Schema gate on load (R-17): <code>trajectory.json/experiment.json</code> are validated against their "0.1" schema on every read; a malformed artifact is a deterministic load error (E-05), not a silent partial read.
I-013	Failure-injection is reproducible: the §17 experiment injects the pinned C-09 <code>parse-config</code> defect via a versioned fixture; the mock-path repair arc is therefore byte-reproducible (I-002) and the iteration-to- VERIFIED count is a <i>pinned</i> constant (3), reproducing not “whatever the model did today.”
I-014	Scope boundary (P1 out-of-scope): the harness is single-agent, single-thread, single-action-batch-per-iteration; subagents (§10), multi-agent orchestration, persistent run-to-run memory, and remote executors are out of scope (referenced, not specified). The <code>OllamaPolicy</code> real path is specified only by interface and is never asserted by the acceptance suite (K-06).

7. Constraints (precise and measurable)

ID	Constraint
K-01	All usage errors (missing flags, bad paths, unknown subcommand) exit 2 (universal).
K-02	<code>run</code> exit: 0 VERIFIED ; 1 BUDGET_EXHAUSTED / STALLED / DENIED_LOOP /terminal ERROR ; 4 PULL_REQUIRED . <code>experiment</code> mirrors <code>run</code> plus 1 on not-reaching- VERIFIED . <code>inspect</code> exit 3 on malformed artifact.
K-03	<code>--max-iterations</code> defaults to a finite $N=8$; any explicit N is a positive integer (0 is a usage error, E-07); the run never runs more than N iterations (I-001).

ID	Constraint
K-04	Sandbox size bound: the sandbox copy is bounded (e.g. a user-configured file-count/byte cap) and the verifier runs in a bounded subprocess (time, output tail length) so a hung or noisy target repo cannot wedge the run. Output tails are length-capped and recorded as such.
K-05	Token / compaction budget: <code>ContextManager</code> enforces a fixed token budget; when <code> C_t > BUDGET</code> compaction fires (R-10) rather than overflowing; the budget and its post-compaction size are recorded in the trajectory.
K-06	Mock path zero-network: the entire automated test suite runs over <code>MockPolicy</code> with no outbound sockets; a socket-opening path in the deterministic core is a source-scan failure (T-02).
K-07	Determinism of surrogate fields: on <code>MockPolicy</code> , <code>tokens.estimated</code> and <code>time_ms</code> are pure functions of the iteration's content and index, via the pinned formula <code>tokens.estimated = len(C_t in chars) + 4 * len(tool_calls)</code> and <code>time_ms = 5 * iteration + len(tool_calls) * 10</code> (F-009: the formula is normative, not exemplified), explicitly labeled synthetic (E-04). Wall-clock timing is NOT reported on the mock path.
K-08	Consecutive-failure budget (F-002): run carries <code>--max-consecutive-errors</code> (default 2) and <code>--max-consecutive-noops</code> (default <code>--max-iterations // 2</code>); K-08 consecutive ERROR verdicts terminate the run ERROR (exit 5), and K-08 consecutive DENY / NOOP /identical-trajectory steps reach DENIED_LOOP / STALLED:NOOP (exit 1). A single error/deny feeds the next iteration without terminating. (This replaces the earlier mis-reference of E-02/E-03/E-09 to K-04.)
K-09	Token/compaction budget unit (F-009): the compact trigger <code> C_t </code> is measured in characters (consistent with the K-07 surrogate), against a pinned BUDGET default (e.g. 8000 chars); on overflow, compaction fires (R-10) unless <code>--no-compact</code> (then E-13 STALLED:BUDGET). Units MUST NOT mix chars and tokens.

8. Edge cases and failure semantics

ID	Case	Semantics
E-01	Task file missing / empty	<code>agent run</code> exits 2 (usage); no trajectory written, no sandbox allocated.
E-02	Policy emits an unrecognizable action (unknown tool, malformed args)	The action is coerced to the ERROR tag (I-004); the iteration records it in errors and the loop continues to the next iteration. After K-08 consecutive ERROR iterations the run terminates ERROR (exit 5, F-003 partition).
E-03	Verifier ERROR (runner missing, import failure, non-reproducible setup)	The <code>Verdict.status</code> is ERROR (distinct from FAILED); the loop <i>continues</i> with the captured output as feedback. K-08 consecutive ERROR iterations terminate the run ERROR (E-02 path; exit 5). A <i>single</i> ERROR never terminates.
E-04	Mock surrogate fields	<code>tokens.mode == "synthetic"</code> and <code>time_ms</code> is the K-07 formula on mock; the real Ollama path reports real counters with <code>mode == "measured"</code> . Mixing is a schema violation (E-05).
E-05	Malformed <code>trajectory.json/experimented.json</code>	<code>inspect/subsequent compare</code> exit 3 (permitted load) with a diagnostic naming the JSON-path, not a silent skip. Bypass with <code>--force</code> only for <code>inspect/experiment</code> (R-17).
E-06	Version mismatch on read	<code>inspect --in x.json</code> on a <code>trajectory_version != "0.1"</code> without <code>--force</code> exits 3 naming the offending field. <code>--force</code> admits it with a banner.
E-07	<code>--max-iterations 0</code>	Usage error (exit 2); the positive-N constraint (K-03) MUST be checked before sandbox allocation.
E-08	<code>--repo</code> points at the bootcamp repo itself or the agent's own source tree	Refused deterministically with exit 2 + a diagnostic (I-011) — a self-editing sandbox would deadlock its own test harness.
E-09	Policy requests a tool not in <code>allow_list</code>	DENY NOT_IN_ALLOWLIST ; no side-effect; recorded in that iteration's errors ; if the policy repeats the same denied call across K-08 consecutive <i>denials</i> the run terminates DENIED_LOOP (exit 1). A single DENY never terminates.

ID	Case	Semantics
E-10	Sandbox root un-writable or non-existent	Sandbox allocation fails; agent exits 5 with a remediation string (check <code>\$TMPDIR</code> , permissions). (<i>F-003: reconciled 4→5 so 4 stays exclusively PULL_REQUIRED.</i>)
E-11	Ollama daemon unreachable on --real	DEGRADED MOCK with distinct banner + trajectory <code>availability_banner</code> : "DEGRADED MOCK: ..."; the run continues on <code>MockPolicy</code> and exits 0 (R-13/R-14).
E-12	--real but qwen3.8 not pulled	PULL_REQUIRED with a remediation string (e.g. <code>ollama pull qwen3.8</code>) + exit 4 (no silent degrade).
E-13	--no-compact under budget pressure	Context overflows the char budget (K-05/K-09); the CLI emits an explicit "budget exceeded, compaction disabled" diagnostic and terminates STALLED:BUDGET (F-004) — never silently truncates.
E-14	edit_file returns applied=false (F-010, C-03)	A failed <code>edit_file</code> (e.g. <code>old</code> not found) is surfaced into the next observation (the repair policy can correct it); it does not advance <code>files_modified</code> , is not a silent no-op, and a run of repeated failed edits feeds the K-08 budget — never a false VERIFIED (§13: only the verifier closes the loop).

9. Acceptance criteria, tests, and evals

9.1 Closed-loop correctness

- **T-01** Over the C-09 *fixture* repo (the `parse-config` task, F-005) with the canonical defect already repaired, the mock agent reaches **VERIFIED** in one iteration and emits a trajectory (C-06) with `final_outcome == "VERIFIED"` and `iterations_used == 1` (1-based, F-001).
- **T-02** A source/graph scan asserts the deterministic core (R-15/I-009) imports **no** model/ network client; `MockPolicy` has zero sockets. (Advisory: no test that opens a socket may run under `pytest`.)
- **T-03** The permission layer, under a default `PermsConfig`, allows `list_files/read_file/ search/edit_file/run_shell` for allowed prefixes and **DENYs** `edit_file/run_shell` on paths outside the sandbox root (I-003) — asserted by both a unit test and by the permission layer rejecting an injected `../escape` tool call.

9.2 Failure injection (§17 / R-11)

- **T-04** The §17 experiment: inject the **C-09 canonical defect** (the `parse_config` blank-line filter mismatch, F-005) into the `fixtures/parse-config/` repo; the mock agent detects the **FAILED** verdict on **iteration 1**, diagnoses via `read_file/search` on **iteration 2**, repairs via `edit_file` on **iteration 3**, and `final_outcome == "VERIFIED"` with `iterations_to_verified == 3` — a *pinned, reproducible* number (I-013), not a model-dependent one.
- **T-05** A *deliberately-misconfigured* policy (a mock that never touches the offending file across `--max-iterations` N iterations) terminates **BUDGET_EXHAUSTED** (exit 1) with `final_outcome != VERIFIED`; the agent does NOT report success it did not achieve (I-006).

9.3 Termination and instrumentation

- **T-06** Over a task where the policy never achieves **VERIFIED**, a run with `--max-iterations 5` stops at the **5th iteration** (1-based, F-001) with `final_outcome == BUDGET_EXHAUSTED` and exit 1; no row past iteration 5 is written into the trajectory (I-001). A terminal core-ERROR run exits 5 (F-003 partition; E-02/E-03/K-08).
- **T-07** `trajectory.json` for T-04 contains exactly the C-06 §17 field set on every iteration row (I-010); a schema-validator test rejects a trajectory missing any required field.
- **T-08** `inspect --in trajectory.json` for T-04 renders a human summary offline (no sockets) and exits 0; a hand-edited (version-bumped) trajectory fails with exit 3 unless `--force` (R-17/E-06).

9.4 Determinism and offline core

- **T-09** Two agent run invocations over the same task/repo/perms with `--mock` produce **byte-identical** `trajectory.json` (I-002); mutating the input changes the output. Tokens and `time_ms` on the mock path carry `mode == "synthetic"` and are the K-07 formula.
- **T-10** A `--repo` pointing at the bootcamp repo or the agent's own source directory is refused (E-08, exit 2).
- **T-11** agent experiment `--mock` over the C-09 fixture produces byte-identical `experiment.json` on rerun (I-013); the `iterations_to_verified` field is the pinned constant 3 across runs.

9.5 Model-availability taxonomy

- **T-12** With Ollama down on `--real`, the run degrades to **DEGRADED MOCK**, banner is recorded in the trajectory (E-11), and exit is 0; with Ollama up but `qwen3.8` absent, exit 4 + remediation string (E-12); with `qwen3.8` present, real mode runs, banner `null`, no `synthetic` tokens.

9.6 Compaction and context engineering

- **T-13** A crafted run whose `C_t` exceeds K-05’s budget *without* `--no-compact` triggers compaction and records the pre/post sizes in the trajectory; with `--no-compact` the same run ends **STALLED** with the explicit diagnostic (E-13). (This test is a *behavioral* check — it asserts the recorded sizes, not that the model “understands” anything.)
-

10. Dependencies and environment

Host prerequisites: Python 3.12, uv, a shell (`bash/zsh`); a git-like layout is not required for the agent itself (only for the bootcamp repo that hosts the lab). Ollama (opt-in, at <http://localhost:11434> with `qwen3.8` pulled) for the real policy path; `DEGRADED MOCK` absorbs its absence, so the offline suite never requires Ollama.

Python packages ([project] in `pyproject.toml`):

- Standard library: `json`, `pathlib`, `subprocess`, `shutil`, `tempfile`, `typing`, `enum`, `dataclasses`, `argparse` (or `click` MAY) — enough for the control loop, the tool controller, the permission layer, the sandbox, the verifier, and the trajectory serializer.
- `jsonschema` — gate `trajectory.json/experiment.json` on read (R-17/I-012).
- Test: `pytest`; `pytest-qt` **only** if the MAY-GUI is implemented (out-of-scope default).
- **No** model/network client in the default `[project]`; the `ollama` (or any HTTP client) lives in an optional extra (`[project.optional-dependencies]` `agents-real`) so that the default install stays mock-only (K-06). This mirrors week-1’s “real = optional, deterministic = default” split.

Build/deploy: `uv venv && uv pip install -e '[agents-real]'` for the full surface; `uv pip install -e .` for the offline core (default). A `Makefile/justfile` MAY expose `make run`, `make experiment`, `make test` (mock only) as convenience wrappers.

11. Traceability matrix (id → where realized)

R-01	10-step closed loop (ch15 S17, S1)	-> <code>control_loop.py::run()</code>	-> T-01, T-04, T-07
R-02	harness as runtime (S2)	-> <code>control_loop.py</code> wiring	-> T-01, T-02
R-03	context engineering (S3)	-> <code>context.py</code>	-> T-13
R-04	tool controller (S4/S6)	-> <code>tools.py</code> / C-03	-> T-03
R-05	permission gate (S11, outside model)	-> <code>permissions.py</code> / C-04	-> T-03, E-09
R-06	verifier closes the loop (S7/S13)	-> <code>verifier.py</code> / C-05	-> T-01, T-04, T-07
R-07	feedback as next observation (S8)	-> <code>control_loop.py</code> FEEDBACK	-> T-04, T-05

R-08	stopping conditions (S8)	-> C-08, loop stop logic	-> T-06
R-09	trajectory instrumentation (S17)	-> instrument.py / report.py	-> T-07, T-09
R-10	compaction (state mgmt, S9)	-> context.py::compact()	-> T-13
R-11	failure-injection experiment (S17)	-> experiment subcommand	-> T-04, T-05, T-13
R-12	sandbox isolation (S12)	-> tools.py + permissions.py	-> T-10, I-003, I-004
R-13	offline determinism (week-1 carried)	-> MockPolicy / instrument	-> T-02, T-09, T-13
R-14	model-availability taxonomy (carried)	-> policy.py::resolve()	-> T-12 (E-11/E-12)
R-15	policy isolation / LLM-free core	-> source-scan / I-009	-> T-02
R-16	CLI primary surface (R-18 SHOULD)	-> cli/agent.py	-> S5.1, T-08
R-17	schema gate on artifact load	-> schemas/*.json, report.py	-> T-07, T-08, E-03
R-18	System-A vs System-B demo (S16)	-> --baseline flag	-> (SHOULD; not gate)
C-01	Task dataclass	-> task.py	-> R-01
C-02	Policy interface	-> policy.py	-> R-02, R-15
C-03	Tool schema (closed set)	-> tools.py	-> R-04, I-003
C-04	Permission decision	-> permissions.py	-> R-05, E-09
C-05	VerifySpec / Verdict	-> verifier.py	-> R-06
C-06	trajectory.json schema	-> schemas/trajectory.json	-> R-09, T-07, T-08
C-07	experiment.json schema	-> schemas/experiment.json	-> R-11, T-04, T-06
C-08	Stopping conditions closure	-> control_loop.py	-> R-08, T-06
C-09	S17 pinned fixture (F-005)	-> fixtures/parse-config/	-> T-04, T-11, I-001
I-001	Bounded loop / termination	-> control_loop.py	-> T-06, K-03
I-002	Byte-deterministic mock path	-> instrument.py + context.py	-> T-09, T-11
I-003	Sandbox isolation	-> tools.py + permissions.py	-> T-03, T-10, E-03
I-004	Closed action/tool spaces	-> C-02/C-03 + coerce	-> E-02
I-005	Context selected not bulk	-> context.py	-> T-13
I-006	Verdict-driven stop only	-> control_loop.py stop gate	-> R-06, T-01, T-02
I-007	Verifier is a runtime signal	-> control_loop.py FEEDBACK	-> T-04
I-008	Permission precedes execution	-> control_loop.py PERMIT	-> R-05, T-03
I-009	Deterministic core LLM/network-free	-> source-scan test	-> T-02, K-06
I-010	Trajectory S17 field totality	-> instrument.py	-> T-07
I-011	Sandbox non-recursive / no self	-> cli::validate_repo	-> E-08, T-10
I-012	Schema gate on load	-> report.py + jsonschema	-> T-08, E-05
I-013	Failure-injection reproducible	-> experiment + C-09	-> T-04, T-11
I-014	Scope boundary / single-agent	-> control_loop.py + I-006	-> (P1 out-of-scope)
K-03	--max-iterations positive	-> cli::parse_args	-> E-07, T-06
K-04	Sandbox/subprocess bounds	-> sandbox.py, verifier.py	-> E-02, E-03
K-05	Context token budget	-> context.py	-> E-13, T-13
K-06	Mock path zero-network	-> [project] default	-> T-02
K-07	Deterministic surrogate fields	-> instrument.py	-> E-04, T-09
K-08	Consecutive-failure budget (F-002)	-> cli::parse_args + loop	-> E-02, E-03, T-13
K-09	Token/compaction unit = chars (F-009)	-> context.py	-> E-13, T-13
E-02	Unrecognizable action -> ERROR	-> policy.py coerce	-> T-02
E-05	Malformed artifact load error	-> report.py load	-> T-08
E-06	Version mismatch	-> report.py load	-> T-08, R-17
E-08	Self-repo / self-source refused	-> cli::validate_repo	-> T-10
E-11	DEGRADED MOCK on Ollama down	-> policy.py::resolve()	-> T-12

E-12	PULL_REQUIRED on missing model	-> policy.py::resolve()	-> T-12
E-14	edit_file applied=false (F-010)	-> tools.py + loop FEEDBACK	-> T-05